



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования «Московский государственный технический университет
имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

Робототехника и комплексная автоматизация

КАФЕДРА

Системы автоматизированного проектирования (РК-6)

ОТЧЕТ ПО ПРЕДДИПЛОМНОЙ ПРАКТИКЕ

Студент

Онюшев Артем Андреевич

(фамилия, имя, отчество)

Группа

РК6-41М

Тип практики

Преддипломная

Название предприятия

АО «Атомстройэкспорт»

Студент

РК6-41М

(Группа)

А. А. Онюшев

(подпись, дата)

Руководитель практики от
кафедры

Ф. А. Витюков

(подпись, дата)

Оценка

Москва, 2026 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный
исследовательский университет)» (МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой РК-6
(Индекс)

А.П. Карпенко
(И.О.Фамилия)

« ____ » _____ 2026 г.

ЗАДАНИЕ

на прохождение производственной практики

Студент Онюшев Артем Андреевич, 2 курса, группы РК6-41М
(фамилия, имя, отчество, № курса, индекс группы)

в период с «09» февраля 2026 г. по «21» мая 2026 г.

Предприятие: АО «Атомстройэкспорт»

Подразделение: Управление разработки платформы Multi-D
(отдел/сектор/цех)

Руководитель практики от предприятия (наставник):
руководитель управления разработки платформы Multi-D,
Барабанов Сергей Валентинович
(фамилия имя отчество полностью, должность)

Руководитель практики от кафедр:
Витюков Фёдор Андреевич, старший преподаватель
(фамилия имя отчество полностью, должность)

Задание.

1. Изучить назначение и роль систем управления доступом в современных информационных системах.
2. Рассмотреть основные модели управления правами доступа.
3. Выполнить сравнительный анализ моделей.
4. Изучить архитектурные принципы и назначение ORY Keto.
5. Рассмотреть способ описания отношений и правил доступа в ORY Keto.
6. Освоить базовую настройку ORY Keto и пример его применения.

Дата выдачи задания «09» февраля 2026 г.

Руководитель практики от предприятия

С. В. Барабанов

(подпись, дата)

Руководитель практики от кафедры

Ф. А. Витюков

(подпись, дата)

Студент РК6-41М
(группа)

А. А. Онюшев

(подпись, дата)

Содержание

Введение	4
Цели и задачи.....	6
Основная часть	7
1. Система управления доступом	7
2. Основные модели.....	10
2.1. Дискреционная модель доступа DAC.....	11
2.2. Мандатная модель доступа MAC.....	12
2.3. Списки контроля доступа ACL	14
2.4. Ролевая модель доступа RBAC	15
2.5. Атрибутивная модель доступа ABAC	17
2.6. Модель доступа на основе отношений ReBAC	19
3. Ory Keto.....	21
4. Настройка Ory Keto	22
Заключение.....	24

Введение

В современных информационных системах вопросы безопасности и корректного разграничения прав доступа играют важную роль. Практически любая прикладная система, будь то корпоративный портал, образовательная платформа, облачный сервис или внутренняя учетная система предприятия, предполагает наличие различных категорий пользователей, каждая из которых должна иметь доступ только к определенному набору функций и данных. Ошибки в проектировании механизмов доступа могут приводить к утечке информации, нарушению целостности данных, снижению надежности системы и возникновению уязвимостей.

По мере усложнения программных продуктов возрастает и сложность моделей управления доступом. Если в простых системах достаточно базового разделения прав по ролям, то в более развитых приложениях требуется учитывать связи между пользователями, объектами и действиями, а также контекст взаимодействия. В связи с этим особую актуальность приобретают современные подходы к авторизации, позволяющие гибко и формально описывать права доступа в рамках сложных предметных областей.

В процессе прохождения практики был выполнен обзор и анализ основных моделей управления правами доступа, используемых в современных информационных системах. Были рассмотрены классические и современные подходы, включая DAC, MAC, ACL, RBAC, ABAC и ReBAC. Особое внимание было уделено relationship-based модели управления доступом, так как данный подход представляет интерес для прикладных веб-систем, в которых права зависят не только от роли пользователя, но и от его связи с конкретным объектом системы.

Кроме теоретического изучения моделей доступа, в ходе практики был рассмотрен инструмент *ORY Keto*, предназначенный для реализации гибкой

модели разграничения прав. Были изучены его архитектурные принципы, способ описания отношений между сущностями, а также базовые подходы к настройке и использованию в прикладных системах. Это позволило перейти от теоретического анализа моделей доступа к их практической реализации.

Актуальность темы практики обусловлена тем, что в современных программных системах, особенно веб-платформах и распределенных сервисах, требуется не только обеспечить аутентификацию пользователей, но и построить надежный, расширяемый и формализованный механизм авторизации. Изучение моделей управления правами доступа и практическое знакомство с инструментами их реализации представляет значительную ценность для последующего проектирования безопасных информационных систем.

Цели и задачи

Цель – изучение моделей управления правами доступа и практическое освоение подходов к реализации системы разграничения доступа с использованием ORY Keto.

Задачи:

1. Изучить назначение и роль систем управления доступом в современных информационных системах.
2. Рассмотреть основные модели управления правами доступа.
3. Выполнить сравнительный анализ моделей.
4. Изучить архитектурные принципы и назначение ORY Keto.
5. Рассмотреть способ описания отношений и правил доступа в ORY Keto.
6. Освоить базовую настройку ORY Keto и пример его применения.

Основная часть

1. Система управления доступом

В современных программных системах управление доступом является одной из базовых составляющих общей архитектуры безопасности. Независимо от предметной области приложения, будь то корпоративная система, облачная платформа, банковский сервис, документооборот или образовательный портал, необходимо определять, какие пользователи могут выполнять те или иные действия и к каким данным они имеют доступ. Такая задача решается с помощью механизмов аутентификации, авторизации и разграничения прав доступа.

Аутентификация представляет собой процесс проверки подлинности пользователя, то есть подтверждение того, что пользователь действительно является тем, за кого себя выдает. Обычно аутентификация осуществляется с помощью логина и пароля, одноразовых кодов, токенов, биометрических данных или иных механизмов подтверждения личности. Однако факт успешной аутентификации сам по себе еще не означает, что пользователю разрешено выполнять любые действия в системе.

После прохождения аутентификации вступает в действие авторизация. Авторизация определяет, какие функции и ресурсы доступны конкретному пользователю. Например, в одной и той же системе один пользователь может только просматривать документы, другой – редактировать их, а третий – управлять правами доступа других пользователей. Таким образом, авторизация отвечает за проверку допустимости совершаемого действия.

Разграничение доступа является более широким понятием и охватывает правила, механизмы и модели, которые используются для назначения, хранения и проверки прав пользователей. По сути, это способ формального описания того, кто, что и при каких условиях может делать в системе. Корректно реализованное

разграничение доступа позволяет не только защищать данные и функции приложения, но и поддерживать целостность бизнес-логики.

Необходимость систем управления доступом обусловлена несколькими причинами. Во-первых, разные категории пользователей работают с системой по-разному и требуют различного уровня полномочий. Во-вторых, с ростом сложности информационной системы увеличивается количество объектов, к которым требуется ограничивать доступ: документы, записи баз данных, проекты, разделы, курсы, уроки, файлы и другие сущности. В-третьих, в современных приложениях часто требуется учитывать не только роль пользователя, но и его отношение к конкретному объекту. Например, один и тот же пользователь может иметь право редактировать собственный курс, но не иметь права редактировать курс другого автора.

В простейших системах разграничение доступа может быть реализовано вручную через условные проверки в коде. Однако такой подход плохо масштабируется, затрудняет сопровождение и приводит к дублированию логики в различных частях приложения. По этой причине в реальных системах применяются формализованные модели доступа, которые позволяют централизованно описывать и проверять права пользователей.

Системы управления доступом должны удовлетворять ряду требований. Прежде всего, они должны обеспечивать безопасность, то есть исключать несанкционированный доступ к данным и функциям. Также они должны быть гибкими, чтобы поддерживать изменение ролей, появление новых сущностей и усложнение предметной области. Кроме того, важны прозрачность и управляемость: разработчик или администратор должен понимать, по каким правилам пользователю предоставляется или запрещается доступ. Наконец, для современных распределенных систем важна масштабируемость, поскольку проверка прав доступа должна оставаться корректной и эффективной даже при большом количестве пользователей и объектов.

Таким образом, системы управления доступом представляют собой важнейший элемент современной информационной системы. Они обеспечивают реализацию бизнес-правил, защиту данных и корректное взаимодействие пользователей с приложением. Для выбора и построения эффективного механизма разграничения прав необходимо рассматривать различные модели доступа и оценивать их применимость в зависимости от особенностей проектируемой системы.

2. Основные модели

В информационных системах применяются различные модели управления правами доступа, отличающиеся способом описания разрешений, степенью гибкости и областью применения. Выбор конкретной модели зависит от типа системы, сложности предметной области, количества пользователей, структуры данных и требований к безопасности. Одни модели хорошо подходят для простых иерархических систем, другие ориентированы на строгое централизованное управление, а третьи используются в современных распределенных веб-приложениях, где доступ зависит не только от роли пользователя, но и от его связи с конкретным объектом.

В рамках практики были рассмотрены основные модели управления доступом: DAC, MAC, ACL, RBAC, ABAC и ReBAC. Каждая из них имеет собственную область применения, а также свои преимущества и ограничения.

2.1. Дискреционная модель доступа DAC

DAC, или Discretionary Access Control, представляет собой дискреционную модель управления доступом, при которой решение о предоставлении доступа к объекту принимается владельцем этого объекта. Иными словами, пользователь, создавший или контролирурующий ресурс, может самостоятельно определить, кто и в каком объеме может с ним работать.

Подобный подход является одним из наиболее простых и исторически ранних. Он широко применялся в файловых системах, где владелец файла может предоставлять или ограничивать доступ другим пользователям. Примером может служить назначение прав на чтение, запись и выполнение файла в операционной системе.

Объект Субъект	O ₁	O ₂	O ₃	O ₄ (Printer)
S ₁	read			
S ₂				Print
S ₃		Read	Execute	
S ₄	read write		read write	

Рис.1 Модель доступа DAC

Преимуществом DAC является простота понимания и реализации. Пользователь получает возможность самостоятельно управлять принадлежащими ему ресурсами, что удобно в небольших и относительно изолированных системах. Однако при усложнении архитектуры приложения такая модель становится менее удобной. Пользователи начинают независимо назначать права, что затрудняет централизованный контроль и может приводить к противоречивым или небезопасным конфигурациям доступа.

Таким образом, DAC хорошо подходит для базовых сценариев управления объектами, но ограниченно применим в системах, где требуется строгая централизованная политика безопасности.

2.2. Мандатная модель доступа MAC

MAC, или Mandatory Access Control, представляет собой мандатную модель управления доступом, при которой правила доступа задаются централизованно и не могут произвольно изменяться пользователями. В отличие от DAC, здесь пользователь не является владельцем политики доступа к ресурсу. Все права определяются системой на основании заранее заданных правил, уровней допуска и классификаций объектов.

Такая модель применяется в системах, где особенно важны строгий контроль и предотвращение несанкционированного распространения информации. Примерами являются военные, государственные и иные защищенные информационные системы, где каждому субъекту и объекту могут быть присвоены уровни секретности, а доступ разрешается только при соблюдении установленных условий.



Рис.2 Модель доступа MAC

Главным преимуществом MAC является высокий уровень формального контроля и предсказуемости политики безопасности. Пользователь не может самовольно изменить параметры доступа, что уменьшает риск ошибок и

злоупотреблений. Однако такая модель сложна в сопровождении и недостаточно гибка для типовых прикладных веб-систем. В пользовательских приложениях, корпоративных порталах и образовательных платформах необходимость в столь жестком централизованном механизме возникает редко.

Следовательно, МАС представляет интерес как фундаментальная модель безопасности, но в практике разработки прикладных систем используется ограниченно.

2.3. Списки контроля доступа ACL

ACL, или Access Control List, представляет собой способ задания прав доступа через список разрешений, связанный с конкретным объектом. В рамках такого подхода у каждого ресурса хранится перечень субъектов и допустимых для них действий. Например, для документа может быть определено, какие пользователи имеют право просматривать, изменять или удалять его.

Формально ACL не всегда рассматривается как отдельная глобальная модель наравне с RBAC или ABAC, однако в прикладной разработке это один из наиболее распространенных механизмов хранения и проверки прав. Он нагляден и удобен для объектов, к которым требуется назначать индивидуальный доступ.

Основное достоинство ACL заключается в прозрачности. Для каждого объекта можно явно увидеть, кто имеет к нему доступ. Это удобно в системах с небольшим числом объектов и пользователей. Однако по мере роста системы возникают сложности: списки становятся длинными, их трудно сопровождать, а права начинают дублироваться между множеством объектов. Кроме того, ACL слабо подходит для описания более сложной логики, когда права зависят от роли пользователя, его отдела, принадлежности к группе или отношений между сущностями.

Таким образом, ACL удобен как локальный механизм описания прав на объект, но в больших системах требует сочетания с более общей моделью управления доступом.



Рис.3 Модель доступа ACL

2.4. Ролевая модель доступа RBAC

RBAC, или Role-Based Access Control, является одной из наиболее известных и широко используемых моделей разграничения доступа. В ее основе лежит идея назначения прав не напрямую пользователям, а ролям. Пользователь, в свою очередь, получает определенную роль, а вместе с ней и соответствующий набор разрешений.

Например, в информационной системе могут существовать роли «администратор», «редактор», «преподаватель», «студент», «модератор». Для каждой роли задается свой набор допустимых действий. Пользователю достаточно назначить подходящую роль, после чего система автоматически определит его полномочия.

Преимуществом RBAC является ее простота, формальная определенность и удобство администрирования. Эта модель особенно эффективна в системах, где набор функций хорошо делится по типовым ролям. Вместо назначения прав каждому пользователю отдельно администратор управляет ограниченным числом ролей, что уменьшает сложность конфигурации.

Однако у RBAC есть и ограничения. В простейшей форме эта модель не учитывает контекст доступа и не отражает отношения пользователя с конкретным объектом. Например, роль «автор» может означать право редактировать курсы, но сама по себе не показывает, какие именно курсы пользователь вправе изменять. В результате в сложных системах приходится дополнять RBAC дополнительными проверками в коде или вводить большое количество специальных ролей, что снижает прозрачность модели.

Таким образом, RBAC остается удобной и практичной моделью для большого числа прикладных систем, но в чистом виде не всегда достаточна для описания сложных объектно-ориентированных прав доступа.

РОЛЕВАЯ МОДЕЛЬ ДОСТУПА (RBAC)

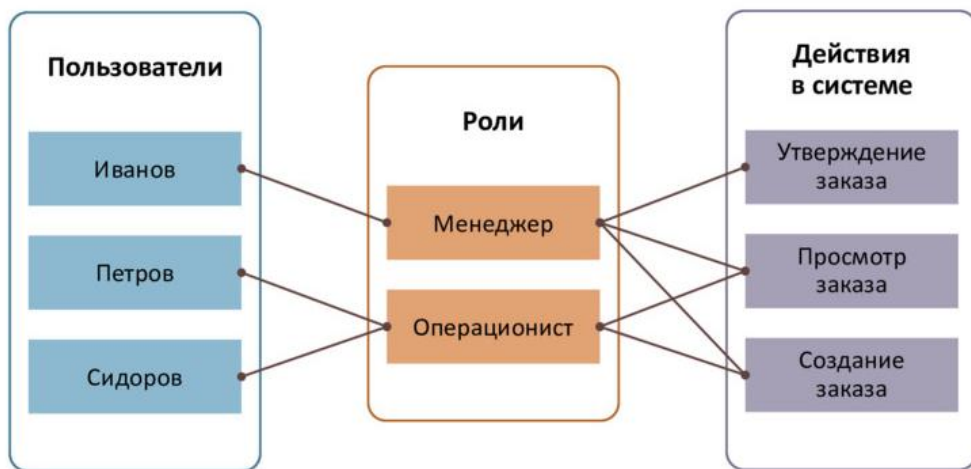


Рис. 4 Модель доступа RBAC

2.5. Атрибутивная модель доступа АВАС

АВАС, или Attribute-Based Access Control, представляет собой модель управления доступом, в которой решение о предоставлении прав принимается на основе набора атрибутов. Атрибуты могут относиться к пользователю, объекту, действию или контексту выполнения запроса.

Например, при использовании АВАС доступ может определяться не только ролью пользователя, но и его подразделением, регионом, статусом, временем обращения, принадлежностью объекта к конкретной категории и иными свойствами. Условие доступа в таком случае задается в виде логического правила. Например: пользователь может просматривать документ, если он работает в том же отделе, что и владелец документа, и документ не имеет ограниченного уровня доступа.

Главным преимуществом АВАС является высокая гибкость. Эта модель позволяет формализовать сложные правила доступа без чрезмерного роста числа ролей. Она особенно полезна в системах, где доступ должен учитывать множество параметров одновременно.

Однако обратной стороной гибкости является повышенная сложность. Политики АВАС труднее проектировать, проверять и сопровождать. При большом числе атрибутов и правил возрастает риск возникновения сложной и плохо читаемой конфигурации, которую трудно анализировать и объяснять администраторам и разработчикам.

Следовательно, АВАС подходит для систем с развитой политикой безопасности и большим количеством контекстных условий, но требует более высокой дисциплины проектирования.

АВАС: СХЕМА ОРГАНИЗАЦИИ ДОСТУПА



Рис. 5 Модель доступа АВАС

2.6. Модель доступа на основе отношений ReBAC

ReBAC, или Relationship-Based Access Control, представляет собой модель разграничения доступа, в которой права определяются через отношения между субъектами и объектами. В данной модели доступ пользователя зависит не только от его роли или атрибутов, но и от того, как он связан с конкретным ресурсом.

Например, в системе пользователь может быть владельцем курса, участником группы, редактором документа, подписчиком проекта или модератором чата. Доступ в таком случае задается не общим правилом вида «все преподаватели могут редактировать все курсы», а более точным отношением: «пользователь может редактировать только тот курс, владельцем или редактором которого он является».

ReBAC особенно удобна для современных прикладных систем, в которых объекты образуют сложные связи: пользователи состоят в командах, группы имеют участников, курсы содержат модули, модули содержат уроки, а права могут наследоваться по иерархии. Такая модель хорошо отражает логику реальных бизнес-процессов и позволяет описывать доступ более естественным образом.

Сильной стороной ReBAC является ее выразительность. Вместо создания большого числа искусственных ролей можно описывать реальные отношения между сущностями. Это делает модель особенно полезной для платформ с большим количеством объектов и зависимых прав. Недостатком является более высокая концептуальная сложность по сравнению с RBAC. Для проектирования и сопровождения такой системы требуется более аккуратное описание доменной модели и правил наследования прав.

Таким образом, ReBAC является одним из наиболее перспективных подходов для современных веб-систем, особенно в тех случаях, когда доступ тесно связан со структурой предметной области.

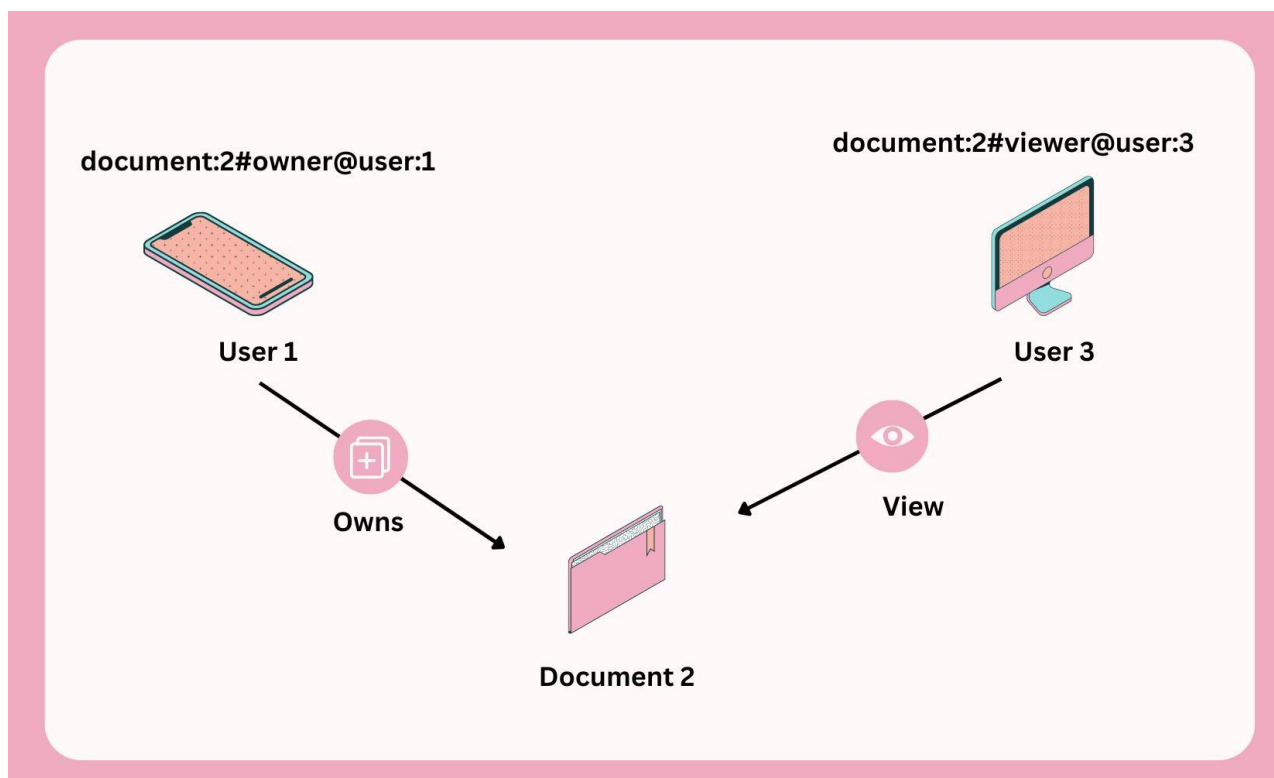


Рис. 6 Модель доступа на основе отношений ReBAC

3. Ory Keto

После рассмотрения основных моделей управления правами доступа особый интерес представляет практическая реализация relationship-based подхода. Для этой цели в рамках практики был изучен инструмент Ory Keto, предназначенный для построения гибкой системы авторизации и разграничения доступа.

Ory Keto представляет собой специализированный сервис управления доступом, предназначенный не для хранения пользовательских учетных данных и не для аутентификации, а именно для проверки прав доступа к объектам системы. Его задача состоит в том, чтобы отвечать на вопросы вида: может ли данный пользователь просматривать, редактировать или удалять конкретный объект. В документации Ory подчеркивается, что отношения являются базовым типом данных Ory Permissions и описывают связи между объектами и субъектами, которые можно рассматривать как ребра графа.

Такой подход позволяет вынести проверку прав из бизнес-логики приложения в отдельный компонент. В результате система становится более прозрачной, а сама логика авторизации может описываться в формализованном виде, а не в виде множества разрозненных условных проверок в коде.

Ory Keto особенно полезен в тех случаях, когда доступ зависит не только от роли пользователя, но и от его связи с конкретным объектом. Например, пользователь может быть владельцем курса, участником группы, редактором документа или модератором эфира. Для подобных сценариев relationship-based модель оказывается более естественной, чем простое назначение ролей.



Рис. 7 Логотип Ory Keto

4. Настройка Ory Keto

После изучения теоретических основ relationship-based модели доступа следующим этапом практики стало ознакомление с базовой настройкой Ory Keto и рассмотрение примеров его использования. Цель данного этапа состояла в том, чтобы понять, каким образом описывается модель доступа, как задаются отношения между сущностями и как на практике выполняется проверка разрешений.

Согласно официальной документации Ory, Ory Keto Quickstart строится вокруг примера видеосервиса, где права доступа определяются через отношения владельца и права просмотра, а сами отношения описываются в виде relation tuples. Такой пример демонстрирует, как Keto используется в роли permission server, а приложение выступает в роли клиента, отправляющего запросы на проверку доступа.

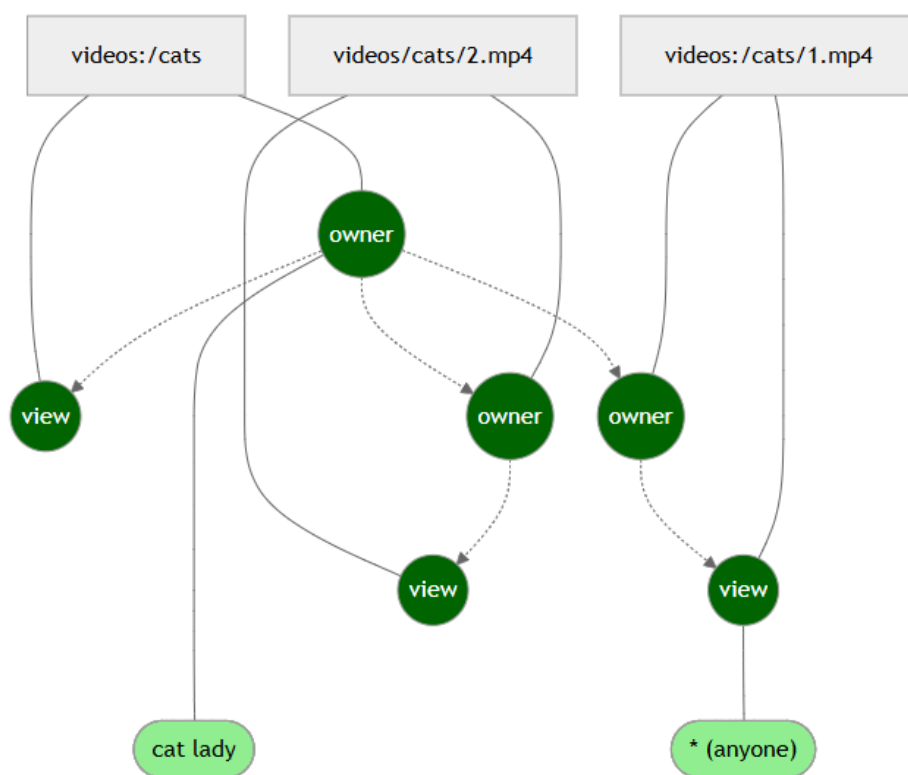


Рис.8 Пример от Ory Keto

Для запуска self-hosted ORY Keto используется конфигурационный файл, обычно называемый keto.yml. В нем задаются параметры подключения к базе данных, адреса read и write API, а также путь к файлу с описанием модели доступа.

Ниже приведен пример базовой конфигурации keto.yml, который подходит для локального запуска и учебного примера с видеосервисом.

Листинг 1 – Пример конфигурации

```
1 dsn: memory/db_url
2
3 serve:
4   read:
5     host: 0.0.0.0
6     port: 4466
7   write:
8     host: 0.0.0.0
9     port: 4467
10
11 log:
12   level: info
13   format: text
14
15 namespaces:
16   - id: 0
17     name: User
18   - id: 1
19     name: Video
```

В данном примере:

- dsn: memory означает использование встроенного in-memory хранилища для учебного запуска;
- serve.read задает read API, через которое выполняются проверки доступа;
- serve.write задает write API, через которое создаются и изменяются отношения;
- namespaces указывает текущие области имен для модели доступа.

Такой вариант удобен именно для учебного или демонстрационного стенда, поскольку не требует отдельной базы данных. Для реальной эксплуатации вместо memory обычно используется постоянное хранилище, например PostgreSQL.

Заключение

В ходе прохождения практики была изучена тема управления правами доступа в современных информационных системах. Были рассмотрены назначение и роль механизмов авторизации, а также основные модели разграничения доступа, применяемые при проектировании прикладных программных систем.

В процессе работы были проанализированы модели DAC, MAC, ACL, RBAC, ABAC и ReBAC. Выполненное сравнение показало, что каждая из них обладает собственной областью применения, преимуществами и ограничениями. Было установлено, что классическая ролевая модель RBAC удобна для систем с фиксированным набором ролей, однако в более сложных предметных областях ее возможностей часто оказывается недостаточно. Модель ABAC обеспечивает высокую гибкость за счет использования атрибутов, но отличается большей сложностью проектирования и сопровождения. Relationship-based модель ReBAC, в свою очередь, позволяет более естественно описывать доступ в системах, где права зависят от связи пользователя с конкретным объектом.

Особое внимание в ходе практики было уделено инструменту ORY Keto как средству реализации relationship-based модели доступа. Были изучены его архитектурные принципы, механизм namespace, relation tuples и способ формального описания модели авторизации. Также была рассмотрена базовая настройка ORY Keto, логика задания отношений между субъектами и объектами и пример проверки прав доступа.

Практическая часть показала, что использование relationship-based подхода и специализированного permission-сервиса позволяет формализовать проверку доступа, сделать систему более прозрачной и упростить сопровождение авторизационной логики в прикладных веб-системах. Это особенно важно для

платформ, в которых присутствует большое количество взаимосвязанных сущностей и сложных сценариев доступа.

Полученные в ходе практики знания и навыки представляют практическую ценность для дальнейшей разработки современных веб-приложений, в том числе образовательных платформ, корпоративных систем и сервисов с развитой объектной моделью. Изучение моделей управления доступом и инструментов их реализации позволило углубить понимание вопросов авторизации и подтвердило значимость правильно спроектированной модели доступа в архитектуре информационной системы.